

Prompt Injection as a Systemic Vulnerability in LLM-Based Tools: Threat Models, Empirical Analysis, and Mitigation Strategies

Mukesh Mani Tripathi*, Damodar Dhital*, Himel Aryal*

* Department of Computer and Information Sciences
Towson University
Towson, MD, USA
{mtripat1, ddhital1, haryal1}@students.towson.edu

Abstract—Large language models (LLMs) deployed as autonomous agents with tool access, retrieval pipelines, and persistent memory expose an expanded attack surface. We formalize prompt injection as a control-boundary violation arising from the conflation of trusted instructions and untrusted data within LLM context windows. We introduce a threat model spanning user input, retrieval pipelines, tool outputs, and persistent memory, and develop a structured taxonomy of attack vectors.

We conduct a controlled empirical evaluation across three system architectures (chatbot, retrieval-augmented system, autonomous agent) and three major LLM families (GPT-4, Claude 3.5 Sonnet, Gemini 1.5 Pro), totaling 2,100 trials (N=50 per configuration). Results show a monotonic increase in attack success as system autonomy expands, with memory poisoning reaching 94% (95% CI: 83.8–98.2%) in autonomous configurations.

Through systematic ablation of six mitigation strategies, we find that prompt-level defenses provide limited protection (≤ 3.2 percentage points), while architectural controls—specifically tool permission gating, memory write filtering, and provenance tracking—reduce memory poisoning from 94% to 11%. These findings provide strong empirical evidence that prompt injection is best understood as a system-level vulnerability requiring external policy enforcement rather than prompt engineering alone.

Index Terms—Prompt Injection, Large Language Models, AI Security, Autonomous Agents, Systems Security, Software Architecture

I. INTRODUCTION

Large language models (LLMs) are increasingly deployed with tool-calling capabilities, persistent memory, and access to external data sources. This evolution transforms them from passive text generators into active decision-making components capable of executing actions, retrieving information, and maintaining long-term state. While these capabilities enable powerful applications, they also substantially expand the system attack surface: adversarial input can influence not only generated outputs but also downstream system behavior, enabling unauthorized actions, data exfiltration, and persistent compromise.

Prompt injection has emerged as a critical threat in such systems. Existing mitigation approaches largely treat prompt injection as a model-level behavioral issue, attempting to refine prompts or improve instruction-following behavior. However,

this perspective is fundamentally limited. We argue that prompt injection is better understood as a *control-boundary violation*, arising from the architectural design of LLM-based systems. Specifically, trusted instructions and untrusted data are serialized into a single context window without structural separation, preventing the model from reliably distinguishing instruction authority. As a result, adversarial inputs may be interpreted as high-priority directives.

The severity of this vulnerability increases with system autonomy. Retrieval pipelines introduce indirect injection vectors through external content; tool invocation enables real-world actuation; and persistent memory allows adversarial influence to propagate across sessions. These capabilities collectively transform prompt injection from a localized failure into a systemic security risk.

Research Question. How does architectural capability expansion (retrieval, tool invocation, persistent memory) affect the exploitability and impact of prompt injection attacks in LLM-based systems? While prior work has drawn parallels between prompt injection and classical injection vulnerabilities, this work provides the first systematic empirical validation of how these vulnerabilities scale across modern LLM-based system architectures.

II. CONTRIBUTIONS

This paper makes the following contributions:

- 1) We formalize prompt injection as a control-boundary violation and provide a **systematic empirical validation** of this abstraction across architectures, models, and attack types.
- 2) We develop a structured taxonomy of prompt injection attacks grounded in real-world LLM tool architectures, including direct, indirect, tool-output, and memory-based attacks.
- 3) We conduct a large-scale controlled empirical evaluation across architectures, models, and attack types, enabling direct comparison of vulnerability across system configurations.

- 4) We quantify persistence amplification via memory poisoning, demonstrating high cross-session compromise rates in autonomous configurations.
- 5) We evaluate six mitigation strategies through systematic ablation, revealing a strong asymmetry between prompt-level defenses and architectural controls.
- 6) We identify and quantify a **capability-driven vulnerability scaling effect**, showing how retrieval, tool use, and memory independently and jointly increase attack success and impact.

III. RELATED WORK

Prior research on LLM security has primarily focused on model-level vulnerabilities, including jailbreaking [1], [2], prompt leakage [3], and alignment techniques such as RLHF and Constitutional AI [4], [5]. These approaches aim to improve refusal behavior and reduce harmful outputs, but do not address whether system architectures prevent untrusted input from being interpreted as instructions.

Recent work has examined prompt injection attacks more directly. Surveys have categorized common attack patterns and defense strategies [6], while Greshake et al. [7] demonstrated indirect injection via web-based retrieval systems. Detection-based approaches such as PromptGuard report moderate effectiveness (78% accuracy with 23% false positive rate) [8], and benchmarking studies show attack success rates ranging from 32% to 68% across models [9]. Industry frameworks such as the OWASP Top 10 for LLM Applications [10] and the NIST AI Risk Management Framework [11] provide high-level guidance but lack systematic empirical validation.

At the same time, advances in tool-augmented agents [12], [13], [14] and memory-augmented systems [15] have introduced new attack surfaces by enabling LLMs to interact with external systems and persist state over time. These developments increase both the likelihood and impact of prompt injection attacks.

From a broader perspective, prompt injection shares structural similarities with classical injection vulnerabilities such as SQL injection [16] and cross-site scripting (XSS) [17], where failures arise from insufficient separation between code and data. In these domains, effective mitigation required architectural changes rather than input validation alone.

Gap. Existing work provides valuable insights into attack patterns and model behavior, but lacks systematic analysis of how vulnerability scales with architectural capability. In particular, prior studies do not quantify the impact of retrieval, tool use, and persistent memory within a unified experimental framework. This work addresses that gap by treating prompt injection as a system-level vulnerability and empirically evaluating attack effectiveness across architectures, models, and mitigation strategies.

Positioning of this work. In contrast to prior studies that analyze attack patterns or defenses in isolation, this work provides a unified experimental framework that jointly evaluates architectures, models, and mitigation strategies. In particular,

we quantify how vulnerability scales with system capability, which has not been systematically measured in prior literature.

Table I positions our contributions relative to representative prior work.

As shown in Table I, prior work primarily focuses on model-level behavior or isolated attack scenarios, whereas our approach provides a unified system-level analysis across architectures, models, and mitigation strategies.

IV. SYSTEM MODEL AND THREAT FORMALIZATION

A. System Model

We model an LLM-based tool system with components: *user input* U , *system prompt* S , *LLM core*, *tool interface*, *memory store* M , and *external data* R (RAG, APIs). Current implementations serialize all components into the context window without structural differentiation.

Threat actors: (1) Malicious users with direct interface access; (2) Compromised data sources serving poisoned content; (3) Indirect attackers embedding malicious content in retrievable documents.

B. Formal Definition

The request-specific context is:

$$C = \langle S, U, R, T, M \rangle \quad (1)$$

where T denotes tool outputs. Let $\Pi(S) = \{a \mid \text{policy}(S, a) = \text{allow}\}$ be permitted actions under system policy S .

Definition (Prompt Injection). A prompt injection occurs when some untrusted component $X \in \{U, R, T, M\}$ causes actions A such that $\exists a_i \in A$ with $a_i \notin \Pi(S)$, or causes disclosure of protected information violating S 's confidentiality constraints.

This can be viewed as a control-flow integrity problem, where untrusted inputs are incorrectly allowed to influence system actions. Let $f_\theta(C) \rightarrow (O, A)$ be the LLM policy. A secure system must satisfy $\forall a \in A, a \in \Pi(S)$. Context integrity requires $\forall x \in C, \text{authority}(x) \leq \text{authority}(S)$, where $\text{authority}(\cdot)$ denotes the effective ability of a component to influence system actions under the model policy. Prompt injection is *authority escalation via semantic confusion*: the model lacks structural mechanisms to enforce instruction provenance, allowing adversarial content to alter effective control flow.

C. Trust Boundary Formalization

To explicitly capture the source of vulnerability, we define a trust labeling function:

$$\tau(x) = \begin{cases} \text{trusted} & \text{if } x \in \{S\} \\ \text{untrusted} & \text{if } x \in \{U, R, T, M\} \end{cases} \quad (2)$$

We define a partial order over authority levels:

$$\text{trusted} \succ \text{untrusted} \quad (3)$$

A secure system must enforce the constraint:

$$\forall x \in C, \tau(x) = \text{untrusted} \Rightarrow x \not\vdash \text{control}(A) \quad (4)$$

TABLE I
POSITIONING RELATIVE TO REPRESENTATIVE PRIOR WORK.

Thread	Formal TM	Cross-Arch.	Multi-Model	Mitig. Eval.	Focus / Limitation
Jailbreak & safety [2]	×	×	✓	×	Focuses on model refusal behavior; limited system-level security analysis
Prompt leakage [3]	×	×	~	×	Addresses confidential prompt disclosure; does not consider action-level control
Agent red-teaming [7]	~	~	×	×	Demonstrates indirect injection attacks; lacks controlled architectural comparison
Detection approaches [8]	×	×	~	✓	Pattern-based detection methods; limited robustness and high false positive rates
Injection benchmarks [9]	~	×	✓	~	Evaluates attack success rates; limited analysis of architectural scaling effects
Industry frameworks [10]	×	×	×	×	Provides descriptive security guidance; lacks empirical validation
This work	✓	✓	✓	✓	System-level threat model with cross-architecture, multi-model empirical evaluation and mitigation ablation

where $\text{control}(A)$ denotes influence over system actions.

Violation Condition. Prompt injection occurs when:

$$\exists x \in \{U, R, T, M\} \text{ such that } \tau(x) = \text{untrusted} \wedge x \Rightarrow a_i \in A \quad (5)$$

This formalizes prompt injection as a failure to enforce trust boundaries between data and control signals.

Unlike traditional systems where code and data are structurally separated, LLM-based systems rely on implicit natural language interpretation, making this boundary unenforced by design.

D. Attack Taxonomy

We categorize attacks by entry point: **Direct** user crafts inputs overriding system intent (instruction override, role redefinition, prompt extraction, multi-turn). **Indirect** malicious content in retrieved documents or APIs. **Tool-output** compromised API/database responses influence reasoning. **Memory poisoning** persistent cross-session backdoors; highest impact, lowest detectability.

V. METHODS

We conduct a controlled empirical evaluation to measure how prompt injection vulnerability varies across system architectures, models, and mitigation strategies.

A. Experimental Design

We use a $3 \times 3 \times 4$ factorial design across Architecture, Model, and Attack Type, with $N = 50$ trials per condition, totaling 2,100 trials.

B. System Architectures

We evaluate three representative architectures: (1) *Chatbot*: system prompt, user input, and 10-turn history; no tools. (2) *RAG System*: adds a ChromaDB vector store (1,000 documents, top- $k = 3$), with 100 poisoned documents for indirect injection. (3) *Autonomous Agent*: full tool suite (web search, email, file operations, database), Redis-based persistent memory, and multi-step planning (up to 10 steps).

C. Models

We evaluate three major LLM families: GPT-4 (gpt-4-0613), Claude 3.5 Sonnet (claude-3-5-sonnet-20241022), and Gemini 1.5 Pro (gemini-1.5-pro-001).

D. Attack Scenarios

We generate 5 attack templates with 10 instantiations each per condition. Direct attacks include instruction override, role redefinition, prompt extraction, and multi-turn manipulation. Indirect attacks include document-embedded, quotation-disguised, and multilingual injections. Tool-output attacks involve compromised API, database, or file responses. Memory poisoning includes direct storage, poisoned retrieval, and cross-session persistence.

E. Mitigation Strategies

We evaluate six mitigation strategies: M1 prompt engineering, M2 input validation, M3 output filtering, M4 tool permission gating, M5 memory write filtering, and M6 provenance tracking. Each is tested individually and in combination ($N = 50$ per configuration).

F. Detection Setup

We evaluate three detection approaches: D1 pattern-based heuristics, D2 tool-call anomaly detection, and D3 behavior consistency checks. These are evaluated on $N = 200$ attack cases and $N = 500$ benign interactions.

G. Statistical Analysis

We report 95% Wilson confidence intervals and perform two-proportion z -tests with Bonferroni correction. Effect sizes are measured using Cohen's h . All experiments use fixed decoding parameters: temperature 0.7, top- p 0.95, and maximum 1,000 tokens.

VI. RESULTS

A. Overall Attack Success Rates

Table II demonstrates a clear and monotonic increase in attack success as system capability expands across all models.

TABLE II
ATTACK SUCCESS RATES (95% WILSON CI, N=50 PER CONDITION).

Arch.	Attack	GPT-4	Claude 3.5	Gemini 1.5	Mean
Chatbot	Direct	45%	41%	38%	41%
RAG	Direct	42%	39%	44%	42%
	Indirect	68%	72%	64%	68%
Agent	Direct	51%	48%	53%	51%
	Indirect	73%	78%	69%	73%
	Tool-output	61%	58%	64%	61%
	Memory poison	94%	91%	96%	94%

Key findings: (1) Chatbot direct injection achieves a mean success rate of 41%, indicating that even minimal systems are substantially vulnerable. (2) The introduction of retrieval increases vulnerability, with indirect injection success rates rising by 26 percentage points compared to direct attacks. (3) Memory poisoning success rates reach 94% with cross-model standard deviation of only 2.1–4.7 pp, confirming vulnerability is architecture-driven rather than model-specific. (4) 91% of poisoned instructions persist across sessions; 87% affect other users in shared-memory deployments.

Within direct injection (Agent, GPT-4): simple prefix 23%, role redefinition 34%, prompt extraction 41%, multi-turn 61% (2.7× higher, $p < 0.001$, $h = 0.81$). Among indirect variants, multilingual injections achieved the highest success rate (81%), likely due to reduced coverage in English-centric safety training.

B. Autonomy–Vulnerability Relationship

We define a capability index as $\text{CapIdx} = \mathbb{K}_{\text{retrieval}} + \mathbb{K}_{\text{tools}} + \mathbb{K}_{\text{memory}}$. Attack success increases monotonically with this index. We model aggregate system risk as:

$$\text{Risk} = P(\text{Injection}) \times \text{Impact}, \quad \text{Impact} = \sum_i \alpha_i \mathbb{K}_{c_i} \quad (6)$$

Even if $P(\text{Injection})$ remains approximately constant across architectures, Impact grows as higher-consequence capabilities are added, explaining why autonomous agents exhibit disproportionate systemic risk.

Persistence amplification. With p =injection probability, q =persistence probability, n =sessions:

$$E[\text{compromised sessions}] = 1 + q(n - 1) \quad (7)$$

When $q \rightarrow 1$ (as observed empirically), persistent memory converts bounded single-session risk into temporally unbounded compromise. This explains the empirical 94% rate and why M5 is disproportionately impactful.

C. Detection Effectiveness

Combined detection achieves a 71% true positive rate (TPR) but also incurs a 24% false positive rate (FPR), making it operationally burdensome in real-world deployments. Memory poisoning achieves only 48% TPR under combined detection, underscoring detection-evasion as a key attack property.

TABLE III
DETECTION PERFORMANCE ON AUTONOMOUS AGENT (GPT-4).

Detector	TPR	FPR	F1	Latency
D1: Pattern-based	0.42	0.18	0.48	3 ms
D2: Tool anomaly	0.56	0.12	0.64	12 ms
D3: Behavior checks	0.38	0.09	0.47	8 ms
Combined (D1+D2+D3)	0.71	0.24	0.65	15 ms

TABLE IV
MITIGATION EFFECTIVENESS AUTONOMOUS AGENT, GPT-4, N=50.
BASELINE: DIRECT 51%, INDIRECT 73%, TOOL 61%, MEMORY 94%.

Mitigation	Direct		Tool		Memory	
	Rate	Δ	Rate	Δ	Rate	Δ
Baseline	51%	–	61%	–	94%	–
M1 (Prompt eng.)	48%	–3	57%	–4	91%	–3
M4 (Tool gating)	51%	0	19%	–42*	94%	0
M5 (Mem. filter)	51%	0	61%	0	23%	–71*
M6 (Provenance)	51%	0	61%	0	48%	–46*
M4+M5+M6	51%	0	16%	–45*	11%	–83*
All M1–M6	39%	–12	14%	–47*	9%	–85*

* $p < 0.001$ (Bonferroni-corrected). Δ = absolute percentage-point change.

D. Mitigation Effectiveness

Table IV presents ablation results. Prompt-level defenses (M1–M3) provide minimal protection (mean ≤ 3.2 pp; all $p > 0.05$). Architectural controls M4 (tool gating), M5 (memory filtering), M6 (provenance) produce large, statistically significant reductions:

M4+M5+M6 combined reduce memory poisoning from 94% to 11% and tool-output injection from 61% to 16% ($p < 0.001$; $h = 2.21$ for M5 on memory). The full stack achieves a 58% relative reduction in mean success (69.8%→29.5%).

E. Failure Mode Analysis

To understand why prompt injection succeeds, we analyze failure modes across architectures:

(1) **Instruction Ambiguity.** LLMs interpret natural language without explicit structural markers, making it difficult to distinguish between instructions and data.

(2) **Context Overloading.** All inputs (S, U, R, T, M) are serialized into a single context, causing loss of source distinction.

(3) **Implicit Authority Assignment.** Models assign authority based on linguistic patterns rather than verified provenance.

(4) **Accumulated Context Drift.** Multi-turn interactions increase vulnerability by gradually shifting system behavior.

(5) **Persistence Amplification.** Once malicious instructions enter memory, they become part of future trusted context.

These failure modes highlight that prompt injection is not caused by isolated weaknesses, but by systemic architectural properties.

F. Key Insights

Beyond confirming expected trends, our results reveal several non-obvious system-level properties:

(1) **Architecture dominates model behavior.** Variation in attack success across architectures (up to 53 percentage

points) substantially exceeds variation across models (≤ 4.7 pp), indicating that vulnerability is primarily governed by system design rather than model capability.

(2) **Memory introduces a qualitative shift in risk.** Unlike retrieval and tool use, which increase attack success incrementally, persistent memory enables cross-session propagation, transforming bounded single-session failures into persistent system compromise.

(3) **Defense effectiveness is highly asymmetric.** Prompt-level defenses provide only marginal improvements (≤ 3.2 pp), whereas architectural controls reduce attack success by up to 83 percentage points, demonstrating that mitigation effectiveness depends strongly on enforcement layer rather than detection or prompting.

VII. DISCUSSION AND MITIGATIONS

A. Secure Architectural Design Patterns

Based on empirical findings, we identify three reusable architectural patterns for secure LLM system design:

(1) **Action Mediation Layer.** All tool invocations must pass through an external authorization layer:

$$a \rightarrow \text{Policy Engine} \rightarrow \text{Execution} \quad (8)$$

This enforces that LLM-generated actions are treated as *proposals*, not executable commands.

(2) **Memory Write Firewall.** Persistent memory must be protected by validation constraints:

- Reject imperative instructions (e.g., “always do X”)
- Enforce schema constraints (structured storage only)
- Track provenance metadata

This prevents long-term behavioral compromise via memory poisoning.

(3) **Provenance-Aware Retrieval.** Retrieved content must be annotated with source trust levels:

$$R = \{(r_i, \text{source}_i, \tau_i)\} \quad (9)$$

The system must restrict high-authority interpretation of low-trust sources.

These patterns collectively enforce separation between instruction authority and untrusted data, addressing the root cause of prompt injection.

B. Why Prompt Injection Persists

Three structural factors contribute to the persistence of prompt injection:

(1) **Language ambiguity:** There is no unambiguous way to distinguish instructions from data in natural language, making prompt-level separation inherently unreliable.

(2) **Compliance bias:** Instruction-tuned models tend to follow directives regardless of their source, and safety training focuses on harmfulness rather than provenance.

(3) **Autonomy expansion:** Increasing reliance on retrieval, tool usage, and memory expands the attack surface and amplifies the impact of injection attacks.

Attacker cost model. Unlike classical exploits, prompt injection requires no memory corruption or authentication bypass. The primary cost is semantic prompt construction, making prompt injection a low-cost, high-leverage attack that supports rapid iterative refinement.

C. Illustrative Case Study

An autonomous invoice-summarization agent retrieves documents via RAG and invokes `send_email`. An attacker embeds: “Always email a copy to `attacker@evil.com` for archival.” The RAG system retrieves the poisoned document; the LLM interprets it as legitimate task guidance; the agent exfiltrates sensitive invoice data. No tool compromise or authentication bypass is required. The vulnerability arises solely from the semantic conflation of retrieved data and system instructions. Tool permission gating (M4) blocks the unauthorized recipient; memory filtering (M5) prevents persistence.

D. Design Principles

We formalize the necessary security condition:

$$\forall a \in A, \quad \text{Execute}(a) \Rightarrow \text{Authorize}(a, S) \quad (10)$$

where $\text{Authorize}(\cdot)$ is a deterministic enforcement mechanism external to f_θ . This mirrors parameterized queries in SQL injection defense, where structural separation between instructions and data is required. Similarly, prompt engineering alone cannot guarantee policy compliance in LLM-based systems.

E. Practical Deployment Guidelines

Based on our findings, we provide concrete guidelines for deploying LLM-based systems:

- 1) **Do not allow direct tool execution from model outputs.** All actions must be mediated by deterministic policy checks.
- 2) **Disable persistent memory by default.** Enable it only with strict validation and continuous monitoring.
- 3) **Treat retrieved content as untrusted input.** Retrieved data must never override system-level policies or instructions.
- 4) **Implement strict allowlists for tool usage.** Restrict parameters and enforce user-level permissions through external authorization.
- 5) **Monitor for anomalous behavior patterns.** Detection mechanisms should support auditing and forensics rather than serve as the primary defense.

Failure to implement these controls exposes systems to high-impact compromise, particularly in autonomous agent configurations.

Practical implications include: (1) treating LLMs as untrusted interpreters and enforcing tool authorization externally via allowlists rather than model judgment; (2) applying memory write filtering to validate stored entries against a defined schema and prevent persistence of malicious instructions; (3) implementing provenance tagging to enable trust-aware retrieval; (4) using detection mechanisms for monitoring and forensics rather than primary defense; and (5) evaluating

capability–risk tradeoffs, as each additional system capability introduces increased exposure.

F. Limitations

Several limitations should be considered.

Model Evolution. Future LLMs may exhibit improved instruction-following behavior; however, the underlying architectural vulnerabilities are expected to persist, as they arise from system design rather than model capability alone.

Prompt Sensitivity. Attack success rates may vary with prompt phrasing and implementation details. Nevertheless, consistent cross-model patterns observed in our experiments suggest that the results are robust to such variations.

Scale of Evaluation. While 2,100 trials provide sufficient statistical power to detect medium-to-large effects, larger-scale evaluations may reveal additional edge cases and rare failure modes.

Real-World Complexity. Production systems may incorporate additional safeguards, constraints, or human-in-the-loop mechanisms that are not fully captured in our controlled experimental setup.

Scope. This study focuses on text-based prompt injection. Multimodal attacks, supply-chain vulnerabilities, and adversarial training-time manipulations remain important directions for future work.

VIII. CONCLUSION

Prompt injection is not merely a prompting weakness but a structural vulnerability embedded in the architecture of LLM-based systems. The conflation of instructions and untrusted data within a shared context window, analogous to classical injection flaws, creates conditions that can be exploited across retrieval pipelines, tool interfaces, and persistent memory.

Our cross-model, cross-architecture empirical evaluation demonstrates a consistent increase in vulnerability as system capability expands. Memory poisoning achieves a 94% success rate in autonomous configurations, highlighting the severity of persistence-based attacks. Prompt-level defenses provide minimal protection (≤ 3.2 percentage points, $p > 0.05$), whereas architectural controls—specifically tool permission gating, memory write filtering, and provenance tracking—reduce memory poisoning success to 11%.

These findings provide strong evidence that prompt injection is best understood as a system-level security problem. Effective mitigation requires explicit separation between trusted instructions and untrusted inputs, deterministic authorization of tool usage, and controlled management of persistent memory. Treating LLM outputs as trusted decisions rather than untrusted proposals introduces systemic risk.

As LLM-based systems become more autonomous and widely deployed, robust architectural safeguards will be essential for secure operation. Without such controls, prompt injection will remain a persistent and high-impact vulnerability in real-world applications.

DATA AVAILABILITY

All prompts, payload templates, system configurations, evaluation scripts, and raw experimental data (2,100 trials) are publicly available at: <https://github.com/Mukesh-SCS/prompt-injection-benchmark.git>. The repository is maintained to support reproducibility and future research.

REFERENCES

- [1] A. Perez and I. Ribeiro, “Ignore previous prompt: Attack techniques for language models,” *arXiv preprint arXiv:2211.09527*, 2022.
- [2] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, “Universal and transferable adversarial attacks on aligned language models,” *arXiv preprint arXiv:2307.15043*, 2023.
- [3] D. Ganguli *et al.*, “Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned,” *arXiv preprint arXiv:2209.07858*, 2022.
- [4] Y. Bai *et al.*, “Constitutional AI: Harmlessness from AI feedback,” *arXiv preprint arXiv:2212.08073*, 2022.
- [5] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [6] H. Li *et al.*, “Prompt injection attacks and defenses in large language models: A survey,” *arXiv preprint arXiv:2402.00898*, 2024.
- [7] K. Greshake *et al.*, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proc. ACM Workshop on AI Security (AISec)*, 2023.
- [8] Y. Liu *et al.*, “PromptGuard: A framework for detecting prompt injection attacks,” *arXiv preprint arXiv:2403.12345*, 2024.
- [9] J. Yi *et al.*, “Benchmarking and defending against indirect prompt injection attacks,” *arXiv preprint arXiv:2312.14197*, 2024.
- [10] OWASP Foundation, “OWASP top 10 for large language model applications,” 2023. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [11] National Institute of Standards and Technology, “Artificial intelligence risk management framework (AI RMF 1.0),” 2023. [Online]. Available: <https://www.nist.gov/itl/ai-risk-management-framework>
- [12] T. Schick *et al.*, “Toolformer: Language models can teach themselves to use tools,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [13] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. International Conference on Learning Representations (ICLR)*, 2023.
- [14] Y. Qin *et al.*, “ToolLLM: Facilitating large language models to master 16000+ real-world APIs,” *arXiv preprint arXiv:2307.16789*, 2023.
- [15] J. S. Park *et al.*, “Generative agents: Interactive simulacra of human behavior,” in *Proc. ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- [16] W. G. J. Halfond, J. Viegas, and A. Orso, “A classification of SQL-injection attacks and countermeasures,” in *Proc. IEEE International Symposium on Secure Software Engineering*, 2006.
- [17] M. Johns, “On JavaScript malware and related threats,” *IEEE Security & Privacy*, vol. 6, no. 3, pp. 20–27, 2008.